# Orders Service

Problem statement, design, failure analysis and the two mock providers

Design summary | October 2026

## 1. Problem Statement

The service exposes **POST /orders** for an e-commerce platform. An order (customer, shipping address, list of items) must be fulfilled from a **single warehouse** that holds every requested product; among the eligible warehouses, the one **closest** to the shipping address is selected. The address is resolved to coordinates by a geocoding API, and payment is submitted to an external payment API (card number, amount, description). Both providers are mocked. The system must be production-ready: a real database and production-grade rigor in data handling.

The core difficulty is that inventory (our database) and money (the payment provider) live in two systems that cannot share a transaction. A failure may occur between any two steps: a process may crash, a remote call may time out after having taken effect, a client may retry, and concurrent requests may contend for the last unit. The design must preserve its invariants under every such interleaving.

## 2. Design Priorities

> **P1. Never sell stock that does not exist.**
>
> **P2. Never hold stock for an order that has been cancelled.**

**P1** is enforced by an atomic conditional update, `UPDATE ... WHERE quantity >= n`, backed by a CHECK constraint that forbids negative stock. Money moves only on *capture*, and capture is attempted only for an AUTHORIZED order, which already holds its stock; hence every charged order has its stock.

**P2** is enforced by releasing the stock in the same transaction that moves an order to a terminal unpaid state (PAYMENT_FAILED or EXPIRED). Known failures release immediately; orders with an uncertain outcome expire after a 120 s deadline plus at most one reconciliation cycle (10 s). The single deliberate exception is the AUTHORIZED order (Section 8).

When the priorities conflict, **P1 prevails**: under uncertainty the system retains stock rather than risk overselling. Section 10 checks both priorities against every combination of outcomes.

## 3. System Overview

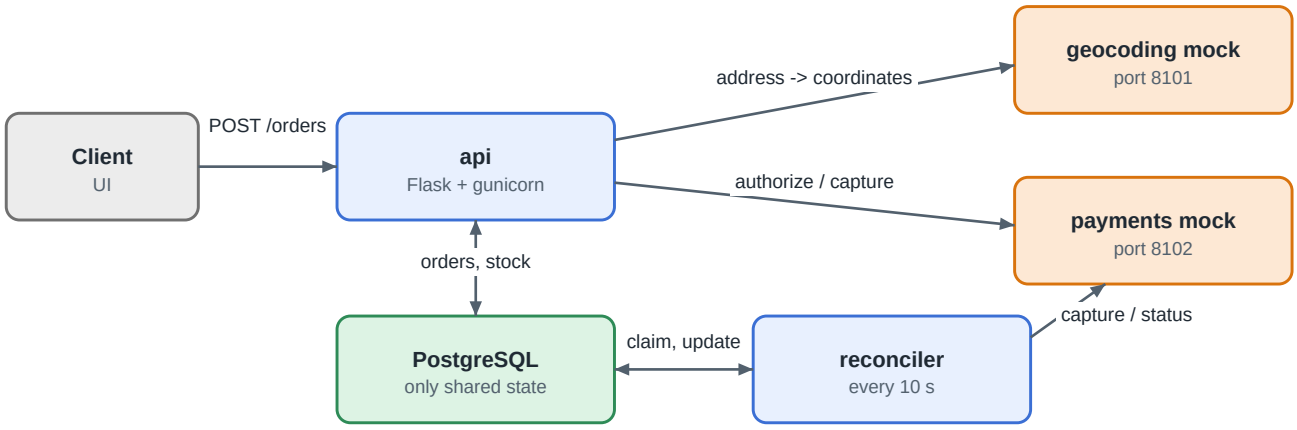
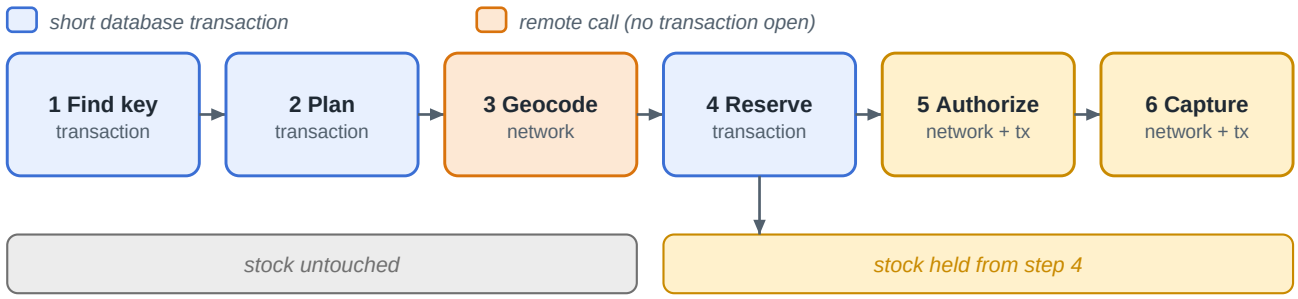



***Figure 1.*** *System components and their interactions.*

The **api** serves requests; the **reconciler** is a separate process that completes the work that requests leave unfinished. PostgreSQL is the only state they share. The geocoder and the payment provider are mocks running in their own containers (Sections 11 to 14).

| Entity | Description |
|---|---|
| **Order** | The purchase: customer, idempotency key, address, selected warehouse, items, total, status, payment id and the deadline for its payment to be authorized. Table `orders`. |
| **OrderItem** | Product, quantity and unit price at purchase time. |
| **OrderEvent** | Append-only log of status changes, written in the same transaction as each change. |
| **Warehouse, Product, Stock** | Reference data loaded at database creation. Stock is the quantity of a product in a warehouse; it is never negative. |
| **Money, Coordinates, OrderStatus** | Value objects: an amount in cents (all prices are in USD), a validated latitude/longitude pair, and the six statuses. |
| **CreateOrder, OrderPaymentService, ReconcileOrders** | The use cases: the POST /orders flow; the payment steps together with every status change and its stock release; and the background job. |

# 4. Order Processing




*Stock returns to the warehouse only if the order ends PAYMENT_FAILED or EXPIRED.*
*If the order ends PAID, the units remain deducted: they were sold.*

***Figure 2.*** *Order processing steps: database transactions (blue) and remote calls (orange).*

Steps 1 and 2 are read-only. If no warehouse holds enough of every product, the request fails with **409 out_of_stock** before any remote call is made. In step 4 a single transaction inserts the order as PENDING and deducts the stock; if the stock changed since the check, nothing is persisted and the next closest warehouse is tried. Steps 5 and 6 call the payment provider and then record the result. **No remote call is made while a transaction is open**, so a slow provider never pins a connection or a lock.

**Listing 1. Stock reservation, step 4 (example: warehouse 2, products 1 and 2)**

```
WITH locked_stock_rows AS (
    SELECT stock.product_id
    FROM   stock
    WHERE  stock.warehouse_id = 2
      AND  stock.product_id IN (1, 2)
    ORDER BY stock.product_id      -- locks taken in a fixed order: no deadlocks
    FOR UPDATE
)
UPDATE stock
SET    quantity = stock.quantity - requested_quantities.quantity
FROM   locked_stock_rows,
       (VALUES (1, 1), (2, 2)) AS requested_quantities (product_id, quantity)
WHERE  stock.warehouse_id = 2
  AND  stock.product_id = locked_stock_rows.product_id
  AND  stock.product_id = requested_quantities.product_id
  AND  stock.quantity >= requested_quantities.quantity   -- short rows are left out
RETURNING stock.product_id;        -- fewer rows than items => reservation failed
```

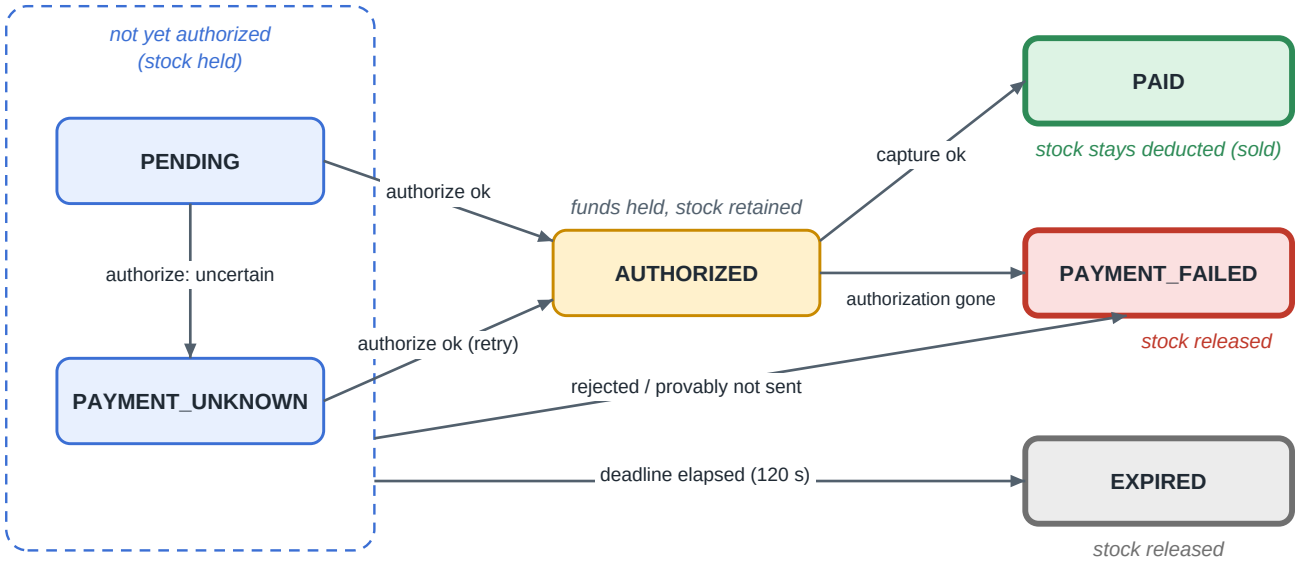
Listing 1 is a single statement. The CTE locks the affected stock rows, and the *UPDATE* deducts each quantity only where `quantity >= requested`. A row without enough stock is silently skipped, so the caller compares the number of rows in `RETURNING` with the number of items: if they differ, it returns without committing, the transaction rolls back (including the `orders`, `order_items` and `order_events` rows inserted just before it), and the next warehouse is tried.

**Idempotency.** Each request carries an `Idempotency-Key`. A unique constraint prevents two concurrent requests with the same key from creating two orders: the one that loses the insert continues with the winner's order. Retrying a terminal order returns it unchanged, retrying an in-progress order resumes it, and reusing a key for a different request yields 422.

***Table 1.*** *Responses of POST /orders.*

| Status | Meaning |
|---|---|
| **201** | Order created and paid. |
| **200** | Replay: this Idempotency-Key already produced an order, which is paid. |
| **202** | Payment still being resolved (PENDING, AUTHORIZED or PAYMENT_UNKNOWN); retry with the same key. Sent with `Retry-After: 5`. |
| **400** | Idempotency-Key missing or not a UUID. |
| **402** | Payment rejected; nothing was charged. |
| **409** | `out_of_stock`: no warehouse can fill the order; or `order_expired`: the order expired before authorization. |
| **422** | Invalid request, unknown product or address, or key reused for a different request. |
| **502** | Geocoder unavailable. |
| **503** | Payment provider unavailable, or service overloaded. |

## 5. Order State Machine




*Thick border = terminal state. AUTHORIZED never transitions to EXPIRED.*

***Figure 3.*** *Order state machine, with the stock effect of each terminal state.*

- **Authorization.** Accepted: AUTHORIZED. Rejected, or provably not sent on the first attempt: PAYMENT_FAILED. Timeout or otherwise uncertain: PAYMENT_UNKNOWN, since the provider may or may not hold the funds (Section 7). A later retry that cannot reach the provider proves nothing and leaves the order PAYMENT_UNKNOWN.
- **Deadline.** An order not authorized within 120 s becomes EXPIRED, performed by the reconciler.

- **Capture.** An AUTHORIZED order becomes PAID, or PAYMENT_FAILED if the provider reports the authorization gone.
- **Concurrency.** Every transition is a compare-and-set under a row lock ("set to X only if the current status is in Y"). The in-flight request, a client retry and the reconciler may race on the same order; exactly one wins each transition, so the stock is released at most once. Transitions outside the diagram are rejected by the domain.

## 6. Stock Lifecycle

**Reserved** at order creation, in the transaction that inserts the order and before any payment call. A single statement attempts every item in the chosen warehouse; if any item is short, the transaction is rolled back, so none stays deducted. Row locks are always acquired in product order, so orders over overlapping products cannot deadlock.

**Listing 2. Order creation and stock reservation: one transaction, four statements**

```
BEGIN;

-- 1. The order, as PENDING. A concurrent duplicate fails here (unique idempotency_key).
INSERT INTO orders (id, customer_id, idempotency_key, request_hash, shipping_address,
                    shipping_latitude, shipping_longitude, warehouse_id, status, total_cents,
                    payment_id, failure_code, expires_at, created_at, updated_at)
VALUES (:id, :customer_id, :idempotency_key, :request_hash, :shipping_address,
        :shipping_latitude, :shipping_longitude, :warehouse_id, 'PENDING', :total_cents,
        NULL, NULL, :expires_at, :created_at, :created_at);

-- 2. Its items, one row per product.
INSERT INTO order_items (order_id, product_id, quantity, unit_price_cents)
VALUES (:order_id, :product_id, :quantity, :unit_price_cents);

-- 3. The first entry of its status history.
INSERT INTO order_events (order_id, from_status, to_status, detail, created_at)
VALUES (:order_id, NULL, 'PENDING', 'created', :created_at);

-- 4. The stock deduction: exactly the statement of Listing 1.
WITH locked_stock_rows AS (...)
UPDATE stock SET quantity = stock.quantity - requested_quantities.quantity ...
RETURNING stock.product_id;

-- Every item returned    => COMMIT: the order and the deduction become visible together.
-- Fewer rows than items  => no COMMIT: ROLLBACK undoes all four, partial deductions included.
COMMIT;
```

Reservation and creation are not a single statement but a single **transaction** of four: the three inserts and the conditional update commit or roll back together, so an order never exists without its stock deducted, nor the reverse. The row locks taken by the CTE in statement 4 are held until `COMMIT` or `ROLLBACK`: a concurrent order over the same products waits, then evaluates `quantity >= requested` against the updated value.

**Retained** while the order is PENDING, PAYMENT_UNKNOWN or AUTHORIZED. Under uncertainty it is never released: the funds may be held, and an authorized order may still be captured.

**Released** only when the order enters PAYMENT_FAILED or EXPIRED, in the same transaction as the status change. This is safe because no capture can occur: there is no payment id, or the provider reports the authorization gone. If the release fails, the status change rolls back with it and the reconciler retries. A PAID order never releases its stock.

## 7. Uncertain Outcomes: PAYMENT_UNKNOWN

When the authorize call times out, fails with a 5xx or returns an unusable body, the provider may or may not have created the authorization, and the response carrying the *payment_id* is lost. The order moves to PAYMENT_UNKNOWN and retains its stock. The system does **not** poll the provider: a lookup requires the payment id, which is precisely what is missing. An order left PENDING by a crash is handled identically.

The uncertainty is resolved in one of two ways:

- **Client retry.** Retrying with the same Idempotency-Key repeats authorize with the same provider-side idempotency key. The provider replays the original authorization, if one exists, and the order proceeds to AUTHORIZED and then PAID. A retry that cannot reach the provider leaves the order PAYMENT_UNKNOWN.
- **Deadline.** If no retry arrives within 120 s, the reconciler moves the order to EXPIRED and releases the stock **without consulting the provider**. This is safe because, lacking a payment id, nobody can capture the payment. If a slow in-flight request obtains the id afterwards, its compare-and-set transition fails and it cancels that authorization.

**Why wait rather than release at the first timeout.** Releasing early would allow the stock to be sold again while the first purchase could still complete, violating P1. **Residual cost:** if an authorization did exist and is never claimed, it remains as a hold on the customer's card, without a backing order, until the provider lapses it (7 days in the mock). Nothing is charged. This trade-off is accepted deliberately in favor of P1.

## 8. AUTHORIZED Orders: Indefinite Retry

An AUTHORIZED order **never expires**: its funds are held and the sale has been accepted, so abandoning it would discard a commitment already made to the customer. The domain forbids the transition AUTHORIZED to EXPIRED; the 120 s deadline applies only to orders that were never authorized.

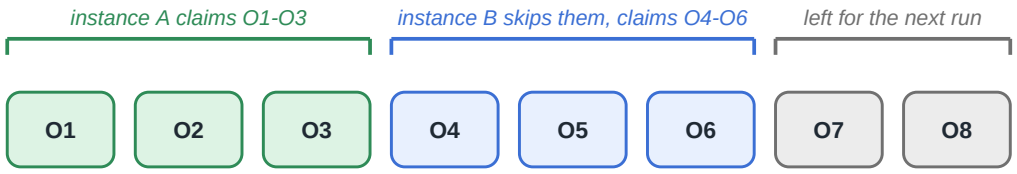
- **Each reconciler run** retries the capture until it succeeds (PAID) or the provider reports the authorization gone (PAYMENT_FAILED with `authorization_canceled` and the stock released).
- The retry is bounded in practice: providers let authorizations lapse after a period (7 days in the mock).
- **Cost:** the stock remains retained while captures keep failing. This is the only case where stock is held for an extended time, so every attempt past the original deadline logs a warning. A hard cap on age was considered and not implemented.
- A client retry also completes the order by going directly to capture; the card number in the retried request is ignored.

## 9. Reconciler and Claims

A separate process that runs every 10 s and completes the work left unfinished by requests. It handles two classes of orders:

- **Overdue and never authorized** (PENDING or PAYMENT_UNKNOWN past `expires_at`): expired with stock release, as described in Section 7.
- **AUTHORIZED and idle for 30 s** (the grace period leaves recent orders to the request still handling them): the reconciler queries the provider by payment id, with no card needed. A captured payment becomes PAID; a canceled one becomes PAYMENT_FAILED; one still authorized is captured. If the provider is unreachable, the order is left for a later run.

**Claims.** Several instances may run concurrently and each provider call is slow, so two instances processing the same order would duplicate remote calls. Each run therefore begins by *claiming* a batch: it selects candidates, writes a lease on each and commits immediately. Afterwards the lease, rather than a row lock, keeps other instances away while the provider is being called.




*Oldest first, FOR UPDATE SKIP LOCKED: rows locked by another instance are skipped, not awaited.*
*Each claimed order gets claimed_until = now + 60 s, committed at once; processing runs outside any transaction.*
*(Batch size is 50; 3 is used only for the drawing.)*

***Figure 4.*** *Work distribution between reconciler instances through claims.*

- A claim changes neither the order's status nor its last-modified time, so the idleness measure is unaffected.
- Deadlines, leases and last-modified times all use the **database clock**, so instances running on machines with skewed local clocks still agree on what is overdue or idle.
- The batch is processed by up to 10 threads; a failing order does not affect the others.
- Leases are never released explicitly; they expire. For an order that could not be completed the lease acts as a back-off before the next attempt, and if an instance dies its orders become claimable again when the lease expires.
- **Correctness does not depend on claims.** They only avoid duplicated work: even if two instances processed the same order, the compare-and-set on the status and the provider-side idempotency keys would keep the outcome consistent.

## 10. Failure Analysis: Verifying P1 and P2

To verify the priorities we enumerated the possible outcomes of each external step (authorize, capture), process crashes between steps, and concurrent actors, and traced every combination to its final state and stock effect. No combination violates P1, and only one (row 11) is exempt from the time bound of P2.

| # | Scenario | Final state | Stock | Why the priorities hold |
|---|---|---|---|---|
| 1 | Authorize succeeds | AUTHORIZED | retained | The stock was deducted before any money can move (P1). |
| 2 | Provider rejects the card | PAYMENT_FAILED | released at once | Nothing was authorized, so no capture is possible (P1); released immediately (P2). |
| 3 | The first authorize attempt never reaches the provider (connect error, circuit open) | PAYMENT_FAILED | released at once | Provably not sent and no earlier attempt exists, so no authorization exists (P1, P2). |
| 4 | Authorize times out, 5xx or unusable response | PAYMENT_UNKNOWN | retained; released only on EXPIRED | An authorization may exist, so no early release (P1). A retry resumes it; otherwise the 120 s deadline bounds the hold (P2). |
| 5 | A retry of a PAYMENT_UNKNOWN order cannot reach the provider (connect error, circuit open) | PAYMENT_UNKNOWN | retained; released only on EXPIRED | An earlier attempt may have authorized, so a retry that was never sent proves nothing: the order is not failed (P1). A later retry resumes it; otherwise the deadline bounds the hold (P2). |
| 6 | Process dies after reserving, or after authorizing but before recording | PENDING | retained until the deadline | Same as 4: no payment id is recorded, so nothing is capturable; expiry or retry resolves it. |
| 7 | An authorization completes after the order already expired | EXPIRED | released once | The transition fails (compare-and-set) and that authorization is canceled; no capture is possible. |

| # | Scenario | Final state | Stock | Why the priorities hold |
|---|---|---|---|---|
| 8 | Capture succeeds | PAID | stays deducted | Units are sold; the stock was retained throughout (P1). |
| 9 | Capture times out or the provider is unreachable | AUTHORIZED | retained | The reconciler queries the provider by payment id: captured becomes PAID, still authorized is retried. Never released while capturable (P1). |
| 10 | Provider reports the authorization gone (canceled, lapsed, capture rejected) | PAYMENT_FAILED | released at once | No capture can occur, so release is safe (P1) and immediate (P2). |
| 11 | Capture fails indefinitely | AUTHORIZED | retained, unbounded | Deliberate exception to P2: the sale is accepted and the funds are held. Bounded by provider lapse; a warning is logged per attempt. |
| 12 | Request, client retry and reconciler act on the same order | one winner | released at most once | Compare-and-set under a row lock: a single transition wins; the others read the stored state. |
| 13 | Reconciler instance crashes, or two instances overlap | unchanged / resumed | unchanged | The lease expires and the batch is claimed again; idempotency keys and compare-and-set prevent a double capture. |
| 14 | Stock release fails | previous state | unchanged | Status change and release share one transaction: both roll back and the reconciler retries. |
| 15 | Two customers request the last unit | one order; the other 409 or next warehouse | deducted once | Atomic conditional UPDATE; the CHECK constraint forbids negative stock (P1). |

**Resulting invariants.** (I1) Stock is released only when an order enters PAYMENT_FAILED or EXPIRED, that is, when no capture can happen: there is no payment id, or the provider reports the authorization gone. Hence a captured payment always has its stock, which establishes **P1**. (I2) Every path that does not end in PAID returns the stock: immediately for known failures, after the deadline for uncertain ones. The only unbounded case is row 11, accepted deliberately; this establishes **P2** up to that exception.

**Residual risks.** A hold on the customer's card without a backing order (rows 4 to 7) until the provider lapses it. A capture confirmed by the provider that cannot be recorded as PAID is logged at CRITICAL for manual refund; it is unreachable under the transitions above and exists as a defensive check.

## 11. Geocoding Mock: Endpoints

A small Flask application that stands in for the third-party geocoder. It runs in the `geocoding-mock` container and is published on `localhost:8101`.

| Endpoint | Behavior and responses |
|---|---|
| **GET /geocode?address=...** | Resolves an address to coordinates.<br>**200** `{"lat": -31.4135, "lon": -64.1811}`<br>**404** `address_not_found`: unknown address<br>**422** `address_required`: no address supplied<br>**503** `service_unavailable`: addresses configured as unavailable |
| **GET /health** | **200** `{"status": "ok"}`. Used by the container health check. |

```
curl "localhost:8101/geocode?address=Bv.%20San%20Juan%20450,%20C%C3%B3rdoba"
{"lat":-31.4135,"lon":-64.1811}

curl -i "localhost:8101/geocode?address=Nowhere%201"
HTTP/1.1 404 NOT FOUND
{"error":{"code":"address_not_found","message":"no results"}}
```

## 12. Geocoding Mock: Dataset and Client Behavior

The mock resolves only the addresses listed in `mocks/geocoding/addresses.json`; matching is case-insensitive and whitespace-normalized. To add an address, edit that file and restart the container.

| Address | Response | Closest warehouse (if it has the stock) |
|---|---|---|
| Av. Corrientes 1234, Buenos Aires | 200 | Buenos Aires |
| Bv. San Juan 450, Córdoba | 200 | Córdoba |
| Bv. Oroño 1000, Rosario | 200 | Rosario |
| Av. San Martín 800, Mendoza | 200 | Mendoza |
| Av. Colón 1500, Mar del Plata | 200 | Buenos Aires |
| Calle Sin Servicio 1, Buenos Aires | 503 | - |
| any other address | 404 | - |

- The service **always** geocodes, even when a single warehouse could fulfill the order, so an invalid address is rejected before anything is reserved.
- A **404** is mapped to **422 invalid_address**. A timeout, connection error or 5xx is retried (2 attempts, exponential backoff with jitter, 3 s timeout) and then reported as **502**. These failures feed a circuit breaker, so during an outage requests fail fast instead of waiting for a timeout. A 404 does not count as a failure: the service responded.
- No database transaction is open during the call.

## 13. Payments Mock: Endpoints

Models a card payments provider with two-phase payments: *authorize* places a hold on the funds and *capture* charges them. It runs in the `payments-mock` container, published on `localhost:8102`. Every POST requires an `Idempotency-Key` header (otherwise `400 idempotency_key_required`). Errors have the shape `{"error": {"code", "message"}}`.

| Endpoint | Behavior and responses |
|---|---|
| **POST /payments** | Creates a payment. Body: `amount_cents` (integer > 0), `currency`, `card_number`, optional `description`, and `capture` (default true; **false only authorizes**).<br>**201** the payment, with status `authorized` or `captured`<br>**402** card declined (`card_declined`, `insufficient_funds`)<br>**422** `invalid_request`, `invalid_card_number` (fails the Luhn check) or `idempotency_key_reused`<br>**503** provider unavailable |
| **POST /payments/{id}/capture** | Captures an authorized payment.<br>**200** the payment, now `captured`<br>**409** `payment_already_captured`, or `payment_canceled` when the hold is gone<br>**404** `payment_not_found` |
| **POST /payments/{id}/cancel** | Voids an authorized payment.<br>**200** the payment, now `canceled`<br>**409** `payment_already_captured` |
| **GET /payments/{id}** | Retrieves a payment: **200** or **404**. An authorization that is never captured lapses and is reported as `canceled`. |
| **POST /payments/{id}/refunds** | Refunds all or part of a captured payment (optional `amount_cents`). **201** the refund; **409** `payment_not_captured`; **422** `amount_too_large`. Not used by the orders service. |
| **GET /health** | **200** `{"status": "ok"}`. |

```
curl -X POST localhost:8102/payments -H "Content-Type: application/json" \
  -H "Idempotency-Key: 5b9c7c5e-1111-4222-8333-444455556666" \
  -d '{"amount_cents": 2500, "currency": "USD", "card_number": "4242424242424242",
       "description": "Order 1", "capture": false}'
{"id":"pay_06991ae108034bafb5e6","status":"authorized","amount_cents":2500,"currency":"USD",
 "description":"Order 1","card_last4":"4242","refunded_cents":0,
 "created_at":"...","expires_at":"... (7 days later)"}

curl -X POST localhost:8102/payments/pay_06991ae108034bafb5e6/capture \
  -H "Idempotency-Key: 5b9c7c5e-1111-4222-8333-444455556667"
{"id":"pay_06991ae108034bafb5e6","status":"captured", ...}
```

## 14. Payments Mock: Idempotency and Test Cards

**Idempotency.** Repeating a request with the same key and body replays the original response instead of re-executing it (the response carries `Idempotent-Replayed: true`). The same key with a different body yields `422 idempotency_key_reused`. This is what allows the service to retry after a timeout without charging twice.

**Lapse and state.** An authorization that is never captured is canceled after a TTL (7 days by default), as on card networks. State is kept in memory; restarting the container resets it. The TTL is configured through `AUTHORIZATION_TTL_SECONDS`.

**Test cards.** Behavior is selected by card number (`scenarios.json`). A "slow" scenario means the operation completes but the response is delayed by 8 s, so the API client times out (5 s) while the payment exists at the provider. Scenarios limited to the first attempts allow a client retry with the same key to succeed.

| Card | Behavior |
|---|---|
| **4242 4242 4242 4242** | Success. |
| **4000 0000 0000 0002** | Declined at authorization: `card_declined`. |
| **4000 0000 0000 9995** | Declined at authorization: `insufficient_funds`. |
| **4000 0000 0000 0101** | Authorize succeeds but responds slowly for the first 2 attempts; a client retry gets the original result. |
| **4000 0000 0000 0200** | Authorize returns 503 for the first 2 attempts; a client retry succeeds. |
| **4000 0000 0000 0309** | Authorize succeeds but every response is late. The order is PAYMENT_UNKNOWN and expires after 120 s, releasing its stock. |
| **4000 0000 0000 0408** | Provider down for authorize: always 503. |
| **4000 0000 0000 0507** | Authorize succeeds; capture responds late for the first 2 attempts. The order stays AUTHORIZED and the reconciler captures it. |
| **4000 0000 0000 0606** | Authorize succeeds; capture is rejected (hold gone). The order fails and releases its stock. |
| **4000 0000 0000 0705** | Authorize succeeds; capture always returns 503. The order stays AUTHORIZED and is retried indefinitely (Section 8). |